

# MiroFish Analysis

## Zenic-Flijo

Analisis Completo del Proyecto

Arquitectura | Bugs | Viabilidad Comercial | Roadmap

Fecha: 10 de Junio, 2026

Motor ORBITAL v3 | Workflow Determinista

# 1. Resumen Ejecutivo

Zenic-Flijo es un sistema de automatizacion de procesos de negocio offline-first, construido sobre un motor determinista circular llamado ORBITAL. El proyecto combina un motor de workflows lineal (viejo) con un nuevo motor orbital (OVC, TOR, RCC, COD, Espectro) que introduce retroalimentacion circular en la ejecucion. Tras un analisis exhaustivo de 100+ archivos fuente, MiroFish identifica 7 bugs criticos, 4 modulos de codigo muerto, 12 carencias competitivas y un roadmap de 8 fases para llevar el producto a mercado.

Veredicto General: El proyecto tiene una base arquitectonica innovadora y solida, pero necesita trabajo significativo en estabilidad del motor ORBITAL, eliminacion de codigo muerto, seguridad, UX y features competitivos antes de ser vendible.

| Metrica            | Valor                                                              | Estado       |
|--------------------|--------------------------------------------------------------------|--------------|
| Archivos Python    | 100+                                                               | Estable      |
| Modulos Core       | 9 (orbital, workflow, nlu, nlp, events, tools, web, data, license) | Completo     |
| Bugs Criticos      | 7                                                                  | Requiere fix |
| Codigo Muerto      | 4 modulos                                                          | Limpiar      |
| Cobertura Tests    | 60+ archivos test                                                  | Bueno        |
| Integridad ORBITAL | Parcial (singleton roto)                                           | Critico      |
| Listo para Vender  | No - requiere 3-4 meses trabajo                                    | En progreso  |

## 2. Arquitectura: Sistema Viejo vs Nuevo

El proyecto evoluciono de un motor lineal (Workflow Determinista v1) a un motor circular (ORBITAL v3). Ambos conviven actualmente, con capas puente que delegan del viejo al nuevo.

### 2.1 Sistema Viejo (Lineal)

El sistema lineal original ejecuta workflows paso a paso: Step1 -> Step2 -> Step3 -> FIN. Sin retroalimentacion, sin retroceso. Los componentes clave son WorkflowEngine, StepExecutor, ConditionEvaluator, BranchHandler, LoopHandler y ErrorHandler. Cada componente opera de forma independiente y secuencial. El EventBus publica eventos pero no hay ciclo cerrado entre la salida y la entrada del sistema.

## 2.2 Sistema Nuevo (ORBITAL - Circular)

El motor ORBITAL introduce un paradigma circular determinista con 5 pilares: OVC (Orbita Variable Circular) gestiona variables con fase, amplitud y velocidad orbital; TOR (Tension Orbital Reciproca) calcula fuerzas entre parejas de variables; RCC (Resonancia de Ciclo Cerrado) detecta cuando las variables resuenan; COD (Colapso Orbital Determinista) converge a estados estables; y EspectroOrbital genera salida multimodal con retroalimentacion. El ciclo completo es: OVC -> TOR -> RCC -> COD -> Espectro -> Retro -> OVC, cerrando el bucle.

## 2.3 Capas Puente (Bridge)

Los componentes del workflow ahora delegan al motor ORBITAL a traves de OrbitalContext (singleton compartido). WorkflowEngine crea variables orbitales para triggers y pasos; StepExecutor calcula TOR entre pasos consecutivos; ConditionEvaluator usa resonancia orbital como fallback; BranchHandler decide ramas por divergencia orbital; LoopHandler converge via COD; y ErrorHandler retroalimenta errores al OVC. El OrbitalContext comparte un unico OVC entre todos los componentes.

| Aspecto           | Lineal (Viejo)         | ORBITAL (Nuevo)                       |
|-------------------|------------------------|---------------------------------------|
| Paradigma         | Step1->Step2->FIN      | OVC->TOR->RCC->COD->Espectro->OVC     |
| Retroalimentacion | Ninguna                | Circular: output modifica input       |
| Variables         | Estaticas (valor fijo) | Orbitales (fase, amplitud, velocidad) |
| Decisiones        | Condiciones booleanas  | Resonancia orbital (TOR)              |
| Bucles            | Contador/condicion     | Convergencia COD (punto fijo)         |
| Errores           | Retry + fallback       | Retroalimentacion negativa al OVC     |

|              |                                 |                                     |
|--------------|---------------------------------|-------------------------------------|
| Determinismo | Si (mismo input = mismo output) | Si (mismas fases = mismo resultado) |
|--------------|---------------------------------|-------------------------------------|

### 3. Bugs Criticos Encontrados

MiroFish detecto 7 bugs criticos que comprometen la integridad del sistema ORBITAL y la seguridad del producto. Estos deben resolverse antes de cualquier release comercial.

#### Bug #1: OrbitalContext Singleton NO comparte OVC con OrbitalEngine

Severidad: CRITICA. OrbitalContext crea su propio OVC y un OrbitalEngine separado con otro OVC. Verificacion en vivo: `id(ctx.ovc) != id(ctx.engine._ovc)` retorna False. Esto significa que las variables creadas en el OVC del Context no son visibles por el Engine. El "OVC compartido" es una ilusion: cada componente esta operando sobre un OVC diferente. Impacto: retroalimentacion rota, TOR calculado sobre variables vacias, RCC sin ciclos, COD sin convergencia real.

#### Bug #2: COD no converge con amplitudes grandes

Severidad: ALTA. En pruebas con amplitudes tipicas (10, 50, 8), COD ejecuta 1000 iteraciones y no converge ( $\delta=0.0796$ ). La causa es que TOR produce valores muy grandes ( $A_i \cdot A_j \cdot \cos = 500$ ) y el `convergence_scale` (0.01) no es suficiente para amortiguar. El sistema necesita normalizacion automatica de amplitudes o adaptive scaling antes del colapso.

#### Bug #3: OrbitalAdapter y OrbitalCompiler crean OVCs independientes

Severidad: ALTA. OrbitalAdapter crea su propio OVC() en `__init__`, y OrbitalCompiler crea su propio OrbitalEngine() en `__init__`. Ninguno usa OrbitalContext. Esto rompe el principio de "OVC compartido" - las variables orbitales de las herramientas y las compilaciones NLU estan aisladas del resto del

sistema.

### **Bug #4: Secrets hardcodedos en config.py**

Severidad: CRITICA (SEGURIDAD). El archivo config.py contiene valores como `SESSION_SECRET = "cambiar_esto_en_produccion_generar_aleatorio_64chars"` y `LICENSE_SECRET_KEY = "cambiar_esto_en_produccion_clave_maestra_hmac"`. En produccion, si el usuario no cambia estos valores (lo cual es el caso mas probable), el sistema es vulnerable a falsificacion de sesiones y licencias.

### **Bug #5: Typo en EventBus - "sbridgeectrum"**

Severidad: MEDIA. En events/bus.py, linea dentro del metodo `publish()`, hay un typo: "sbridgeectrum" en lugar de "spectrum". Esto causaria un `KeyError` al intentar acceder al dato en el callback del subscriber.

### **Bug #6: OrbitalContext.engine y OrbitalContext.ovc no estan sincronizados**

Severidad: ALTA. Cuando el `WorkflowEngine` llama a `self._ctx.engine.run_tick()`, este engine opera sobre su propio OVC interno, no sobre el OVC del `OrbitalContext`. Las variables inyectadas via `_inject_trigger_as_orbital()` van al `ctx.ovc`, pero el engine no las ve. El tick orbital se ejecuta sobre un OVC vacio.

### **Bug #7: BusinessTools usa eval() en \_tool\_calculate**

Severidad: CRITICA (SEGURIDAD). Aunque restringe `__builtins__`, `eval()` con input del usuario es inherentemente inseguro. Un usuario malicioso puede inyectar codigo Python a traves de expresiones peligrosas. El archivo `workflow/tools.py` es codigo muerto pero si se usa, es una vulnerabilidad critica.

## **4. Codigo Muerto y Sobrantes**

MiroFish identifico 4 modulos de codigo muerto que deben eliminarse o archivar para reducir la superficie de ataque y la confusion en el codebase.

| Modulo                                                           | Estado    | Razon                                                           | Accion     |
|------------------------------------------------------------------|-----------|-----------------------------------------------------------------|------------|
| workflow/tools.py<br>(BusinessTools)                             | Obsoleto  | Reemplazado por src/tools/ con servicios CRM, Invoice, etc.     | Eliminar   |
| nlu/preprocessing.py,<br>rule_matcher.py,<br>rule_normalizer.py  | Obsoleto  | Reemplazados por DDE v3 pipeline (nlu/ completo)                | Eliminar   |
| config/database.py,<br>config/settings.py,<br>config/__init__.py | Obsoleto  | Reemplazados por config.py unificado y data/database_manager.py | Eliminar   |
| nlu/pipeline.py<br>viejo (13-stage lineal simple)                | Duplicado | Existe pipeline.py DDE v3 que reemplaza la version simple       | Consolidar |

## 5. Integridad del Sistema ORBITAL

El motor ORBITAL es la innovacion central de Zenic-Flijo. Su integridad es critica para la propuesta de valor del producto.

### 5.1 Pilares ORBITAL - Estado por Componente

| Pilar | Funcionalidad                  | Tests   | Integridad                                     |
|-------|--------------------------------|---------|------------------------------------------------|
| OVC   | Gestion de variables orbitales | OK      | Buena                                          |
| TOR   | Tension orbital reciproca      | OK      | Buena                                          |
| RCC   | Deteccion de resonancia        | OK      | Buena                                          |
| COD   | Colapso determinista           | Parcial | Problemas - no converge con amplitudes grandes |

|                   |                             |          |                                     |
|-------------------|-----------------------------|----------|-------------------------------------|
| Espectro          | Salida multimodal + retro   | OK       | Buena                               |
| OrbitalContext    | Singleton compartido        | FALLANDO | ROTO - OVC no compartido con Engine |
| OrbitalEngine     | Coordinador de 5 pilares    | OK       | Buena                               |
| OrbitalDB         | Persistencia SQLite orbital | OK       | Buena                               |
| OrbitalAdapter    | Envoltorio orbital de tools | Parcial  | Problemas - OVC independiente       |
| OrbitalCompiler   | Compilacion orbital NLU     | Parcial  | Problemas - Engine independiente    |
| OrbitalRepository | Conversion lineal->orbital  | Parcial  | Aceptable                           |

## 5.2 Problema Central: El Singleton Roto

El problema mas critico del sistema ORBITAL es que el OrbitalContext no cumple su promesa de "OVC compartido". La arquitectura intenciona que todos los componentes (WorkflowEngine, StepExecutor, ConditionEvaluator, BranchHandler, LoopHandler, ErrorHandler, EventBus) compartan un unico OVC a traves de OrbitalContext. Sin embargo, OrbitalContext.\_\_init\_\_() crea un OVC propio Y un OrbitalEngine con otro OVC interno. Ademàs, OrbitalAdapter y OrbitalCompiler crean sus propios OVCs independientes. Resultado: el sistema tiene 4+ OVCs aislados en lugar de 1 compartido.

Solucion propuesta: OrbitalContext debe crear un UNICO OVC y pasarlo a OrbitalEngine. OrbitalAdapter y OrbitalCompiler deben usar OrbitalContext en lugar de crear instancias propias. Esto requiere refactorizar OrbitalEngine para aceptar un OVC externo en su constructor, y modificar todos los consumidores para obtener sus pilares del OrbitalContext.

## 6. Analisis de Viabilidad Comercial

Zenic-Flijo compete en el mercado de automatizacion de procesos para PYMES. Su propuesta de valor unica es: offline-first, pago unico, y privacidad total. Sin embargo, el mercado actual exige features que el producto aun no tiene.

## 6.1 Competidores Directos

| Producto          | Precio          | Offline | Privacidad | NLU         | Tools       |
|-------------------|-----------------|---------|------------|-------------|-------------|
| Zapier            | \$600/ano       | No      | No         | Basico      | 6000+       |
| Make (Integromat) | \$360/ano       | No      | No         | No          | 1500+       |
| n8n (self-hosted) | Gratis/\$360    | Parcial | Si         | No          | 400+        |
| Huginn (OSS)      | Gratis          | Si      | Si         | No          | Limitado    |
| Zenic-Flijo       | \$399 (una vez) | Si      | Si         | Si (DDE v3) | 9+4 integr. |

## 6.2 Ventajas Competitivas Actuales

Zenic-Flijo tiene 3 ventajas reales sobre la competencia: (1) Precio: \$399 una vez vs \$360-2400/ano de suscripciones, ahorrando al cliente \$300-2000 en el primer ano. (2) Offline-first: funciona sin internet, lo cual es critico para PYMES en zonas con conectividad inestable. (3) Privacidad: todos los datos quedan en la computadora del cliente, sin servidores de terceros. Esto es especialmente valioso para empresas reguladas (GDPR, HIPAA, leyes locales de proteccion de datos). Ademas, el motor ORBITAL es una innovacion unica en el mercado: ningun competidor ofrece retroalimentacion circular determinista en su motor de ejecucion.

## 6.3 Desventajas Competitivas Actuales

Las desventajas principales son: (1) Solo 9 herramientas nativas + 4 integraciones vs 6000+ de Zapier. (2) La UI web es basica - Flask templates sin framework moderno. (3) No hay version movil ni API publica documentada. (4) El motor ORBITAL tiene bugs criticos de integridad. (5) No hay marketplace de templates ni comunidad. (6) El instalador no esta probado en produccion. (7) No hay



sincronizacion cloud opcional. (8) El NLU tiene solo 5 intents configurados vs decenas que tendria un producto comercial.

## 7. Sistemas que Debes Agregar para Ser Vendible y Competitivo

MiroFish identifica los siguientes sistemas criticos que faltan en el proyecto, organizados por prioridad comercial. Sin estos, el producto no puede competir seriamente en el mercado actual de automatizacion.

### 1. Dashboard en Tiempo Real con WebSocket [Prioridad: CRITICA]

El dashboard actual usa polling HTTP. Los competidores muestran actualizaciones en tiempo real. Necesitas Flask-SocketIO para notificar al frontend cuando un workflow completa, falla, o un evento ocurre. Sin esto, el usuario debe refrescar manualmente la pagina para ver cambios.

### 2. UI Moderna con Framework Frontend [Prioridad: CRITICA]

La UI actual es HTML/CSS/JS vanilla con Flask templates. Para ser competitivo, necesitas React o Vue.js con un design system profesional (Tailwind + Headless UI). El chat NLU debe parecer Slack/ChatGPT, no un formulario basico. El editor visual de workflows debe ser drag-and-drop con nodos conectables.

### 3. Marketplace de Templates [Prioridad: ALTA]

Zapier tiene 6000+ integraciones y templates prehechos. Necesitas al menos 50 templates profesionales (registro de clientes, facturacion automatica, alertas de stock, onboarding de empleados, reportes semanales, etc.) que el usuario pueda activar con 1 clic. AutoPilot service existe pero tiene solo 10 templates genericos.

### 4. API Publica Documentada (OpenAPI/Swagger) [Prioridad: ALTA]

Los desarrolladores quieren integrar Zenic-Flijo en sus propios sistemas. Necesitas una API REST documentada con OpenAPI 3.0, SDKs para Python y JavaScript,

y webhooks salientes. Sin API publica, el producto es una isla que no se conecta con nada.

## 5. Sistema de Plugins/Extensiones [Prioridad: ALTA]

No puedes construir todas las integraciones tu solo. Necesitas un sistema de plugins que permita a terceros crear herramientas (ej: plugin de Stripe, plugin de Shopify, plugin de WhatsApp Business). El sistema actual de tools es monolitico - cada tool esta hardcodeada en el codigo fuente.

## 6. Sincronizacion Cloud Opcional [Prioridad: MEDIA]

Aunque la propuesta de valor es offline-first, muchos clientes quieren backup automatico en la nube y sincronizacion entre dispositivos. Implementa sincronizacion cifrada end-to-end con un servidor minimal (opcional, el usuario elige activarlo). Sin esto, pierdes clientes que necesitan trabajar en multiples computadoras.

## 7. Mobile Responsive / PWA [Prioridad: MEDIA]

Los duenos de PYME gestionan sus negocios desde el celular. La UI actual no es responsive. Necesitas Progressive Web App (PWA) con notificaciones push para que el usuario reciba alertas de stock bajo, facturas vencidas, etc. directamente en su telefono.

## 8. Observabilidad y Monitoreo [Prioridad: MEDIA]

El sistema necesita metricas en tiempo real: workflows ejecutados/dia, tasa de exito, latencia promedio, errores por herramienta. Actualmente hay logging basico pero no hay dashboard de monitoreo, alertas proactivas, ni health checks automaticos.

## 9. Multi-tenencia Mejorada [Prioridad: MEDIA]

El sistema tiene RBAC basico (admin, editor, viewer) pero no tiene multi-tenencia real. Para el tier Revendedor (\$1,499), necesitas que cada cliente tenga su propio espacio aislado con sus propios workflows, datos y configuracion.

## 10. Integraciones Adicionales (Priority Stack) [Prioridad: ALTA]

Las integraciones mas demandadas que faltan: Stripe/PayPal (pagos), Shopify/WooCommerce (ecommerce), Google Sheets bidireccional, Microsoft 365, QuickBooks (contabilidad), Twilio (SMS), y WhatsApp Business API completo. Actualmente solo tienes Gmail, Sheets, Telegram y Slack.

#### 11. Editor Visual de Workflows Drag-and-Drop [Prioridad: CRITICA]

El editor actual (editor.html/editor.js) es basico. Necesitas un editor de nodos visual tipo n8n/Node-RED donde el usuario arrastra herramientas, conecta pasos con flechas, configura parametros inline, y ve el flujo completo. Esto es el feature #1 que compran los usuarios no-tecnicos.

#### 12. Sistema de Auditoria y Compliance [Prioridad: MEDIA]

Para empresas reguladas, necesitas: audit trail inmutable (quien hizo que y cuando), exportacion de logs para compliance, retencion de datos configurable, y data classification (sensible, publico, interno). El audit\_log actual existe pero es basico.

## 8. Roadmap de Implementacion Recomendado

MiroFish recomienda el siguiente roadmap priorizado para llevar Zenic-Flijo de su estado actual a un producto vendible y competitivo. Cada fase tiene dependencias claras y entregables verificables.

| Fase | Nombre                    | Duracion  | Entregables                                                                          | Prioridad |
|------|---------------------------|-----------|--------------------------------------------------------------------------------------|-----------|
| 1    | Fix Criticos              | 2 semanas | Fix OrbitalContext singleton, COD convergence, secrets, codigo muerto, typo EventBus | URGENTE   |
| 2    | UI Moderna + Dashboard    | 4 semanas | React + Tailwind, WebSocket dashboard, editor visual drag-and-drop v1                | CRITICA   |
| 3    | Templates + Integraciones | 3 semanas | 50 templates, Stripe, WhatsApp Business, Google Sheets bidireccional                 | ALTA      |

|   |                             |           |                                                                        |       |
|---|-----------------------------|-----------|------------------------------------------------------------------------|-------|
| 4 | API Publica + Plugins       | 3 semanas | OpenAPI 3.0 spec, SDK Python/JS, sistema de plugins con loader         | ALTA  |
| 5 | Cloud Sync + PWA            | 2 semanas | Sync E2E cifrado, PWA con push notifications, responsive mobile        | MEDIA |
| 6 | Observabilidad              | 1 semana  | Metrics dashboard, alertas proactivas, health checks, uptime monitor   | MEDIA |
| 7 | Compliance + Multi-tenencia | 2 semanas | Audit trail inmutable, data classification, tenant isolation           | MEDIA |
| 8 | Pulido + Lanzamiento        | 2 semanas | Testing E2E, performance, documentacion, landing page, installer final | ALTA  |

Tiempo total estimado: 19 semanas (4.5 meses) para un producto vendible y competitivo. Las fases 1-2 son bloqueantes y deben completarse antes de cualquier demo o venta.

## 9. Tonos Importantes – Lo que Debes Saber

MiroFish destaca los siguientes tonos criticos que determinan el exito o fracaso comercial del producto.

### 9.1 El Motor ORBITAL es tu Diferenciador #1

Ningun competidor tiene un motor circular determinista. Zapier es lineal. Make es lineal. n8n es lineal. Tu motor ORBITAL con retroalimentacion es una innovacion real que puede posicionarse como "el motor que aprende de si mismo". Pero SOLO si funciona correctamente. Un motor orbital con bugs es peor que un motor lineal que funciona. Prioriza la integridad del motor sobre cualquier feature nueva.

### 9.2 El Precio es tu Arma Principal

\$399 una vez es extremadamente competitivo. Pero ese precio solo funciona si el producto tiene valor percibido. Un producto con 9 herramientas y UI basica NO justifica \$399. Necesitas al menos 50 templates profesionales, un editor visual

pulido, y integraciones con herramientas que los PYME ya usan (Stripe, WhatsApp, Google). El valor percibido no viene del motor ORBITAL (que el usuario no ve) sino de los templates y la experiencia de uso.

### **9.3 El Offline es tu Moat Defensivo**

La promesa "funciona sin internet" es poderosa para PYMES en América Latina, África y Asia donde la conectividad es inestable. Pero el offline-first también es una limitación: los usuarios esperan sincronización entre dispositivos. La solución es "offline-first con cloud opcional" - el producto funciona 100% sin internet, pero si el usuario quiere, puede activar sync cifrado. Esto te da lo mejor de ambos mundos.

### **9.4 La NLU es tu Segunda Diferenciación**

Poder crear workflows hablando en español es un feature poderoso. Pero la NLU actual tiene solo 5 intents básicos (registro\_cliente, factura, stock\_bajo, notificación, general). Para ser competitivo, necesitas al menos 30-50 intents cubriendo: envío de emails programados, reportes automáticos, aprobación de pedidos, seguimiento de clientes, recordatorios, importación de datos, exportación a Excel, alertas de vencimiento, cálculo de comisiones, etc. Cada intent es un workflow preconstruido que el usuario activa hablando.

### **9.5 El Editor Visual es el Feature que Vende**

El 80% de los usuarios de automatización no son programadores. Quieren ver su workflow como nodos conectados con flechas, no como JSON. n8n y Make tienen editores visuales excelentes. Tu editor actual es funcional pero no es competitivo. Invertir en un editor visual tipo React Flow o Vue Flow es la inversión con mayor retorno en ventas.

### **9.6 El Installer Define la Primera Impresión**

Si el instalador falla o es confuso, el cliente pide reembolso. PyInstaller y Nuitka generan falsos positivos de antivirus frecuentemente. Necesitas: (1) Firmar el ejecutable con un certificado de código, (2) Testear en Windows 10/11 limpios,

(3) Incluir un instalador GUI profesional (NSIS o Inno Setup), (4) Probar el primer-run experience: login -> primer workflow -> ejecución exitosa en menos de 3 minutos.

## 10. Recomendaciones Finales de MiroFish

Resumen de las acciones mas importantes, ordenadas por impacto en la viabilidad comercial del producto.

| #  | Accion                                                     | Impacto                                         | Esfuerzo  |
|----|------------------------------------------------------------|-------------------------------------------------|-----------|
| 1  | Fix OrbitalContext singleton (OVC compartido real)         | Critico - sin esto el motor ORBITAL no funciona | 2 dias    |
| 2  | Fix COD convergence (adaptive scaling)                     | Alto - colapso orbital es fundamental           | 1 dia     |
| 3  | Eliminar codigo muerto (tools.py, viejo NLU, config viejo) | Medio - reduce confusion y superficie de ataque | 1 dia     |
| 4  | Generar secrets aleatorios en primera ejecucion            | Critico - seguridad basica                      | 0.5 dias  |
| 5  | Editor visual drag-and-drop (React Flow)                   | Critico - feature que vende                     | 3 semanas |
| 6  | 50 templates profesionales con 30+ intents NLU             | Alto - valor percibido del producto             | 2 semanas |
| 7  | API publica con OpenAPI + SDK Python/JS                    | Alto - permite ecosistema de terceros           | 2 semanas |
| 8  | WebSocket dashboard + notificaciones push                  | Alto - experiencia de usuario moderna           | 1 semana  |
| 9  | Integraciones: Stripe, WhatsApp Business, QuickBooks       | Alto - las mas demandadas por PYMES             | 2 semanas |
| 10 | Cloud sync opcional E2E cifrado                            | Medio - amplía mercado a multi-device           | 1 semana  |

Conclusions de MiroFish: Zenic-Flijo tiene una base arquitectonica unica y un diferenciador real (motor ORBITAL + offline-first + NLU en espanol). Pero la implementacion actual tiene bugs criticos de integridad en el motor ORBITAL, carencias significativas en UI/UX, y pocas integraciones. El camino a un producto vendible requiere 4-5 meses de trabajo enfocado: primero estabilizar el motor, luego construir la experiencia de usuario, y finalmente expandir el ecosistema de integraciones y templates. La prioridad #1 absoluta es fixear el OrbitalContext singleton - sin un OVC realmente compartido, todo el sistema ORBITAL es decoracion.